Understanding ArrayList

How to create an **`ArrayList` of primitive types (using wrapper classes)** and **`ArrayList` of object types (user-defined class)** in Java.

✅ 1. `ArrayList` of Primitive Type (Wrapper class)

Java does not allow primitive types (like `int`, `double`) directly in an `ArrayList`. You must use their **wrapper classes** (like `Integer`, `Double`).

Java

```
import java.util.ArrayList;

public class PrimitiveArrayListExample {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>(); // for
int values

        numbers.add(10);
        numbers.add(25);
        numbers.add(30);

        System.out.println("Integer List: " + numbers);

        ArrayList<Double> prices = new ArrayList<>(); // for
double values

        prices.add(12.5);
        prices.add(99.9);

        System.out.println("Double List: " + prices);
    }
}
```

✅ 2. `ArrayList` of Object Type (User-defined class)

Example using a custom `Student` class.

```java
import java.util.ArrayList;

class Student {
    String name;
    int marks;

    public Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    public void display() {
        System.out.println(name + " scored " + marks + "
marks.");
    }
}

public class ALExample {
    public static void main(String[] args) {
        ArrayList<Student> studentList = new ArrayList<>();

        studentList.add(new Student("Ali", 85));
        studentList.add(new Student("Sara", 90));

        for (Student s : studentList) {
            s.display();
        }
    }
}
```

How **not using type safety** in `ArrayList` (i.e., **raw type usage**) affects your Java code and why **casting is necessary** in such cases.

🎓 Case Study: `Student` Class (Recap)

Java

```
class Student {
    String name;
    int marks;

    public Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    public void display() {
        System.out.println(name + " scored " + marks + "
marks.");
    }
}
```

❌ Example Without Type-Safety (Raw `ArrayList` — Needs Casting)

Java

```
import java.util.ArrayList;

public class NoTypeSafeExample {
    public static void main(String[] args) {
        ArrayList studentList = new ArrayList(); // ❌ No
<Student> — not type-safe

        studentList.add(new Student("Ali", 85));
        studentList.add(new Student("Sara", 90));
```

```
        for (int i = 0; i < studentList.size(); i++) {
            // ❗ Must cast to Student, or you'll get a
compile-time error
            Student s = (Student) studentList.get(i); // ✅
Casting
            s.display();
        }
    }
}
```

⚠️ What happens if we forget to cast?

Java

```
Object s = studentList.get(i);
s.display();  // ❌ Compile-time error: cannot find symbol
```

Because `studentList.get(i)` returns an `Object`, and you cannot call `display()` on a generic `Object`.

✅ Example With Type-Safety (`ArrayList<Student>` — No Casting Needed)

Java

```
import java.util.ArrayList;

public class TypeSafeExample {
    public static void main(String[] args) {
        ArrayList<Student> studentList = new ArrayList<>(); // ✅
Type-safe

        studentList.add(new Student("Ali", 85));
        studentList.add(new Student("Sara", 90));

        for (Student s : studentList) {
```

```
            s.display(); // ✅ No casting required
        }
    }
}
```

🧠 Summary: Why Prefer Type-Safe `ArrayList<Student>`

Without Type-Safe	With Type-Safe (`<Student>`)
Needs explicit casting	No casting required
More error-prone	Compiler catches type errors
Slower to debug	Easier to read and maintain

To **emphasize the relationship** between **`ArrayList` implementation** and **class relationships** such as **Association** and **Aggregation** by:

🧩 Conceptual Understanding

🔹 **Association:**

- A **"uses-a"** relationship.
- A class contains a reference to another class.
- **May use `ArrayList` to store multiple references.**

✅ **ArrayList is commonly used in Association** when a class temporarily uses or works with multiple instances of another class.

```java
class Teacher {
    String name;
    public Teacher(String name) { this.name = name; }
}

class Department {
    ArrayList<Teacher> teachers = new ArrayList<>(); //
Association

    public void addTeacher(Teacher t) {
        teachers.add(t); // using Teacher
    }
}
```

🧠 **Key Point**: `Department` **uses** `Teacher`. There's no ownership or lifecycle dependency — if `Department` is destroyed, `Teacher` objects can still exist elsewhere.

🔸 **Aggregation:**

- A **"has-a"** relationship with **independent lifecycles**.
- One class **contains** a collection of another class via `ArrayList`.
- If the container is destroyed, the contained objects can still exist.

✅ **ArrayList expresses Aggregation well**, since it allows you to model containment of multiple objects that are created independently.

```java
class Student {
    String name;
    public Student(String name) { this.name = name; }
}
```

```
class Course {
    String courseName;
    ArrayList<Student> studentList; // Aggregation

    public Course(String courseName, ArrayList<Student>
studentList) {
        this.courseName = courseName;
        this.studentList = studentList;
    }

    public void displayStudents() {
        for (Student s : studentList) {
            System.out.println(s.name);
        }
    }
}
```

🧠 **Key Point**: The `Course` **has-a list** of `Student` objects. The `Student` objects may also be used by other courses (shared ownership, not exclusive).

✅ How to Emphasize

Strategy	Explanation
💬 A real-world analogies	E.g., A `Library` has a list of `Books`. Books exist independently (Aggregation).
🛠️ Show with `ArrayList<Class>`	Implement examples like `ArrayList<Student>` in `Course`, `ArrayList<Doctor>` in `Hospital`.
👨‍🏫 Highlight lifecycle behavior	Ask: “If the container object is deleted, should the contents be deleted too?”
🧪 Use debugging/code tracing	Let students trace object references in memory to observe shared vs. exclusive ownership.

🔗 Additional Support from GFG Link

The link provided:
https://www.geeksforgeeks.org/association-composition-aggregation-java/

It contains diagrams and code examples for:

- Association (loose relationship)
- Aggregation (has-a, independent)
- Composition (has-a, dependent)